ASTANA IT UNIVERSITY · SOFTWARE ENGINEERING · PRE-DEFENSE 1

AI-based generation of software test cases from requirements documents using a local language model

Team 5 · group SE-2405

Team

Nuraly Abat · Sultan Muratbek · Kaisar Yessenov

Supervisor

Almas Ospanov

Date

24 October 2026

The problem

Test design is manual

QA engineers read long requirement texts and write test cases by hand. Quality depends on the person and the deadline.

Gaps surface late

A requirement without a test is usually found when a defect reaches production, the most expensive moment.

Cloud AI is not allowed

Banks, public sector and telecom cannot send internal requirements to external AI services.

A second issue: vague requirements such as “the system responds quickly” produce plausible but useless tests, so requirement quality must be checked before generation.

Aim and measurable outcomes

Build a self-hosted system that turns an SRS document into structured, traceable test cases, checks requirement quality first and keeps a human reviewer in control.

≥ 80%
requirements covered by at least one test

≥ 70%
cases accepted by a reviewer with no or minor edits

≥ 95%
outputs valid against the JSON schema

≤ 10%
cases with values not in the source requirement

Must scope: document upload and parsing · requirement extraction · quality gate (ISO/IEC/IEEE 29148) · generation · review · traceability matrix · export to CSV, XLSX, Gherkin, JSON

Evidence: three practitioner interviews

P1 · AI product lead, quasi-public IT

P2 · QA engineer, telecom

P3 · QA engineer, e-commerce

Online chat, 5 questions about past cases, no product pitch. Notes anonymised in the repository.

Hypothesis	Status
H2 Gaps in coverage are found late	Verified, 3 of 3
H3 Cloud AI is restricted	Verified, 2 strong + 1 partial
H4 Vague requirements hurt tests	Verified, 2 strong + 1 partial
H5 AI output needs review	Verified, 3 of 3
H1 Test design takes much time	Partial
H6, H7 Export formats, languages	Partial, 1 of 3
H8 Test design is the main pain	Refuted, 2 of 3

WHAT PRACTITIONERS TOLD US

“Corporate and confidential data cannot simply be sent to external public models. We deploy our own models on our own servers, in a closed environment.”

P1, AI product lead · translated from Russian

P3: the security department forbids uploading requirements to public clouds under NDA. This is the gap commercial tools leave open.

Literature review and research gap

27

publications, critically appraised

3

standards and guidance: ISO/IEC/IEEE 29148, 29119-4, OWASP Top 10 for LLM

Includes the supervisor's 10 required recent papers: Q1, Q2, Q3 journals and the Scientific Journal of Astana IT University.

What the literature shows

- LLMs can draft useful test cases, but correctness still needs human review.
- Vague or low-quality requirements reduce generation quality.
- Most studies use large cloud models and English requirements.

Gap we address

No solution combines schema-validated cases, fully on-premise open-weight models, a 29148 quality check before generation and requirement-level traceability with measured hallucination rates. We combine them and measure the result on English and Russian requirements.

Existing systems

System	Where the model runs	Human review	Input
Xray (Jira)	Vendor cloud	Yes	Jira requirement issues
TestRail 9.5	Cloud only	Yes	Requirement text
Qase AIDEN	Cloud, external LLM API	Yes	Text or Jira/GitHub issue
Tricentis qTest Copilot	Not stated	Yes	Requirements, images
ChatGPT, manual	Vendor cloud	Copy-paste	Pasted text
Our system	Organisation’s own server	Mandatory, logged	Whole SRS document

“AI generates test cases” is not new. Our difference: self-hosting, a quality gate before generation, a test design technique and requirement ID on every case, and published quality metrics.

Technology decisions

Decision	Choice	Main reason
D1 LLM serving	Ollama (dev), vLLM (prod)	Simple local setup; high throughput in production
D2 Backend	Python, FastAPI	Async API; Pydantic maps to the JSON schema
D3 Storage	PostgreSQL + pgvector	Relational links and vector search in one database
D4 Long jobs	Postgres job queue + SSE	No extra broker; live progress for the user
D5 Frontend	Next.js, TypeScript	Rich tables for review and traceability
D6 Model family	Qwen3, Gemma, Mistral	Chosen by benchmark in December 2026

Each decision compares two or three alternatives against requirement IDs and is recorded as an ADR in the repository.

Requirements

14

functional requirements

23

non-functional requirements

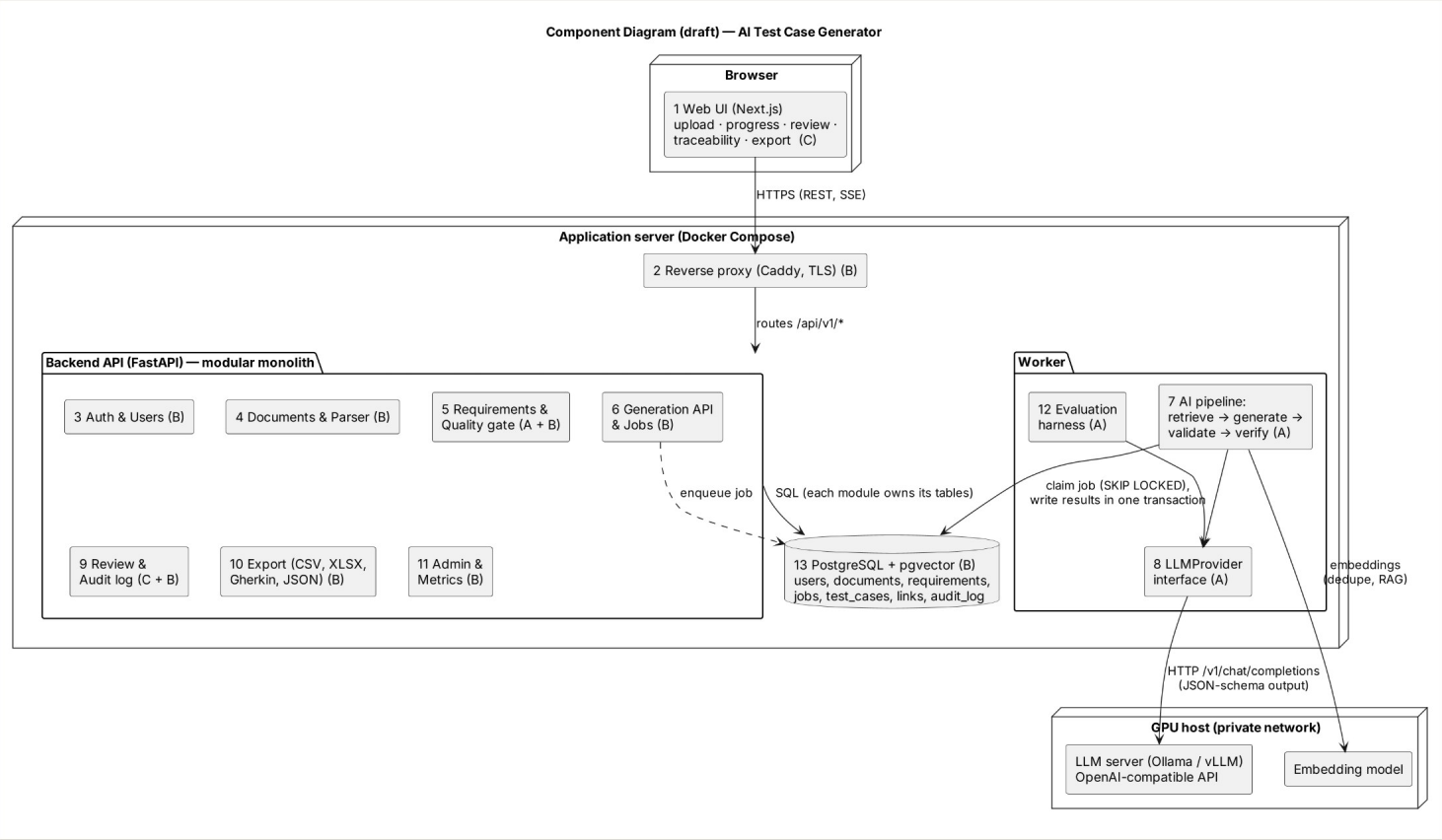
Every Must requirement has an acceptance criterion; priorities follow MoSCoW.

Must have

1. Upload and parse PDF, DOCX and Markdown
2. Extract individual requirements
3. Quality gate: ambiguity and incompleteness checks (ISO/IEC/IEEE 29148)
4. Generate structured test cases with a technique per case
5. Review: accept, edit or reject each case
6. Traceability matrix and coverage
7. Export to CSV, XLSX, Gherkin and JSON

Key constraint C-01: documents never leave the organisation’s infrastructure.

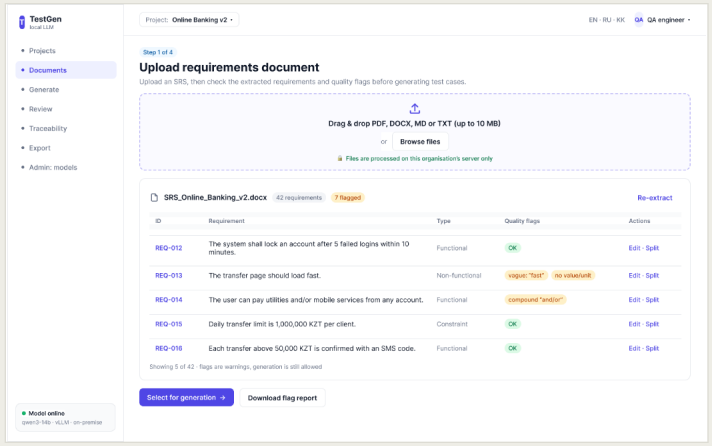
Architecture



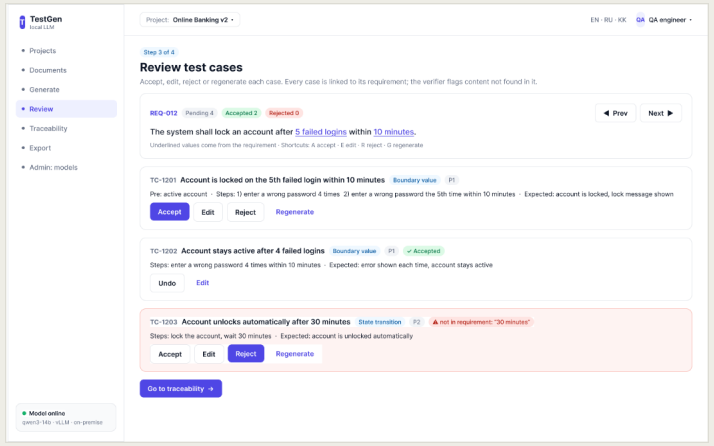
Main flow

1. Upload SRS
2. Parse and extract requirements
3. Quality gate flags issues
4. Worker generates cases with RAG
5. Verifier removes duplicates and ungrounded cases
6. Reviewer accepts, edits or rejects
7. Matrix and export

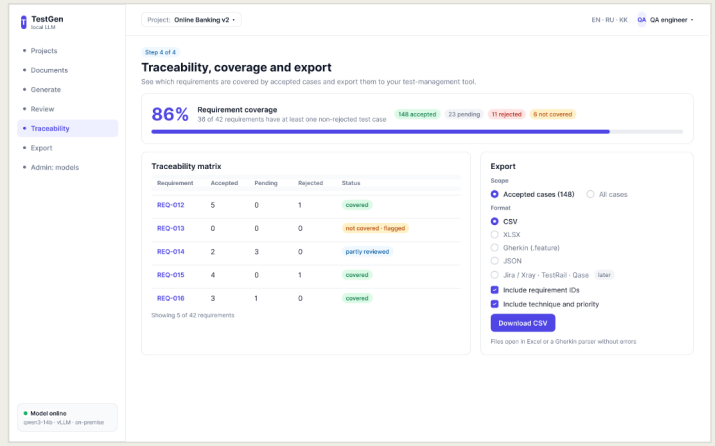
UX prototype



W1 · Upload and quality check



W3 · Review of generated cases



W4 · Traceability, coverage, export

Clickable Figma prototype of four screens. Every case shows its requirement ID and technique; nothing is saved without a reviewer’s decision.

How we will prove it

Configurations

ID	Setup
B0	Manual test design (human baseline)
B1	Same model, zero-shot, no schema
P1	Few-shot + JSON schema
P2	P1 + RAG context
P3	P2 + verifier (proposed)

Measured on

- Public requirements documents with known licences and gold test cases
- English and Russian requirements
- Coverage, acceptance rate, schema validity, hallucination and duplicate rates
- User study: time to an accepted suite versus B0, SUS ≥ 68
- Load, reliability and security scenarios

Team and process

Member	Role	Delivered
Nuraly Abat	A · AI pipeline, evaluation	Proposal, literature review, LLM serving ADR, evaluation plan, interview notes
Sultan Muratbek	B · Backend, data, DevOps	Interview materials, technology ADRs, requirements, context and component diagrams
Kaisar Yessenov	C · Frontend, UX, QA	Repository, README, competitor analysis, evidence synthesis, use case diagram, prototype

38
tasks in Notion, IDs DIP-n

22
pull requests reviewed by another member

17
supervisor findings logged with owners

Status, limitations and next steps

Open limitations

- Three interviews, no bank respondent
- Project framing after H8 to agree with the supervisor
- Prototype not yet tested with users
- GPU and local model smoke test pending
- ADRs still Proposed; no code yet, as planned

Towards pre-defense 2

When	What
31 Oct	GPU plan, Ollama smoke test
Nov 2026	Interviews P4–P5, user test, code skeleton, dataset
Dec 2026	First end-to-end generation, benchmark round 1
31 Jan 2027	MVP with all Must requirements

Thank you. Questions?

Repository · github.com/yssnov/ai-testcase-generation (tag prob-period-1)

Prototype · figma.com/design/m2NAfVfFTXCA4SMN7yZp5F

Nuraly Abat · Sultan Muratbek · Kaisar Yessenov · SE-2405